

PHANTOM PROTOCOL

Technical Specification № 2

Sphinx⁺ Extended Packet Format & Zero-Knowledge Verifiable Shuffle

March 2026 · Draft v0.1

ABSTRACT

This specification defines two tightly coupled components of Phantom Protocol: (1) Sphinx⁺, an extended cryptographic packet format derived from the original Sphinx design by Danezis and Goldberg (2009), augmented with post-quantum key encapsulation, a per-batch commitment header, and a shuffle-witness anchor enabling downstream proof verification; and (2) the Phantom ZK Shuffle — a non-interactive zero-knowledge argument of correct mixing, based on the Bayer-Groth verifiable shuffle argument (EUROCRYPT 2012), adapted for the Sphinx⁺ packet structure and the real-time constraints of a live mix network. Together these components provide the cryptographic guarantee that no mix node can reorder packets dishonestly, drop packets, duplicate packets, or forward them in input order — and that any such misbehaviour is immediately detectable by any network participant, without revealing anything about which input maps to which output.

Contents

1. Background and Motivation
2. Sphinx Original — Recap and Limitations
3. Sphinx⁺ — Extended Packet Format
4. The Phantom ZK Shuffle
5. Proof Generation Protocol
6. Proof Verification Protocol
7. Batch Lifecycle — End to End
8. Security Analysis
9. Performance Characteristics
10. Rust Implementation Guide
11. Test Vectors
12. References

1. Background and Motivation

The critical weakness of all currently deployed mix networks — Katzenpost and Nym included — is that mixing is unverified. Relay nodes are trusted to shuffle packet batches honestly. A compromised or malicious node can simply forward packets in the same order it receives them, completely defeating the anonymity guarantee of the mix layer, and no observer — including other nodes or the protocol developers — can detect this.

This is not a theoretical concern. In a threat model that includes nation-state adversaries, legal compulsion of node operators is routine. An adversary that legally compels 3 out of 8 nodes in a stratified mix layer to forward without shuffling can deanonymize a significant fraction of traffic even without breaking any cryptographic primitive.

Phantom Protocol eliminates this attack surface entirely. Every mix node must produce a publicly verifiable zero-knowledge proof that its output batch is a correct permutation of its input batch — with each packet properly decrypted. The proof reveals nothing about the permutation. An invalid or missing proof causes immediate ejection of the node from the network. A passing proof guarantees correct mixing even if the node operator is actively hostile.

Design Objective

Sphinx⁺ and the Phantom ZK Shuffle together must satisfy: (a) every packet is processed correctly at every hop, provably; (b) no information about the shuffle permutation is revealed by the proof; (c) proof generation and verification are fast enough for real-time mix network operation; (d) the construction is compatible with post-quantum key encapsulation.

2. Sphinx Original — Recap and Limitations

The original Sphinx packet format (Danezis & Goldberg, IEEE S&P 2009) is the gold standard for mix network packet design. It provides:

- ▶ Bitwise unlinkability: a node cannot link its input packet to its output packet — the packet header is blinded at each hop.
- ▶ Path length hiding: no node knows its position in the path or the total path length.
- ▶ Tagging attack resistance: packet modification is detectable via per-hop MACs.
- ▶ Replay attack resistance: nodes maintain a seen-packet cache to reject replays.
- ▶ Compact format: the header grows linearly with path length, not quadratically.

2.1 Original Sphinx Structure

A standard Sphinx packet is a triple $(\alpha, \beta, \gamma, \delta)$ where:

```

 $\alpha$  =  $g^x$  (group element, blinded per-hop)
 $\beta$  = routing_info || filler (onion-encrypted routing data)
 $\gamma$  =  $\text{MAC}(\mu_k, \beta)$  (per-hop integrity MAC)
 $\delta$  =  $E(\pi_k, \text{payload})$  (onion-encrypted payload)

At each node  $n_i$ :
   $s_i = \alpha^{\{sk_i\}}$  (shared secret via DH)
   $\mu_i = \text{KDF}(s_i, \text{'mac\_key'})$ 
   $\pi_i = \text{KDF}(s_i, \text{'payload\_key'})$ 
   $b_i = \text{KDF}(s_i, \text{'blind'})$  (blinding factor)
   $\alpha_{i+1} = \alpha^{b_i}$  (blinded for next hop)
   $\beta_{i+1} = \text{shift\_left}(\beta, 2\kappa) \text{ XOR } \text{PRG}(\pi_i)$ 

```

2.2 Limitations of Original Sphinx

Limitation	Impact on Phantom Protocol
Classical DH only (X25519)	Vulnerable to quantum adversary with Shor's algorithm. Harvest-now-decrypt-later attacks are active today.
No batch-level commitment	Nodes cannot commit to their input batch before processing — prerequisite for the ZK shuffle proof.
No shuffle witness	The packet carries no information enabling a verifier to confirm it was part of a correctly shuffled batch.
No proof anchor field	No field exists in the header to carry or reference the batch shuffle proof.
Fixed group element size	Kyber ciphertext (~1568B) cannot be accommodated without header restructuring.

Sphinx⁺ addresses all five limitations while preserving the full security properties of the original format.

3. Sphinx⁺ — Extended Packet Format

Sphinx⁺ is a backwards-incompatible extension of the Sphinx packet format designed for Phantom Protocol. It introduces three structural additions: a hybrid post-quantum key encapsulation block, a batch commitment anchor, and a shuffle proof reference field. All existing Sphinx security properties are preserved.

3.1 Packet Layout

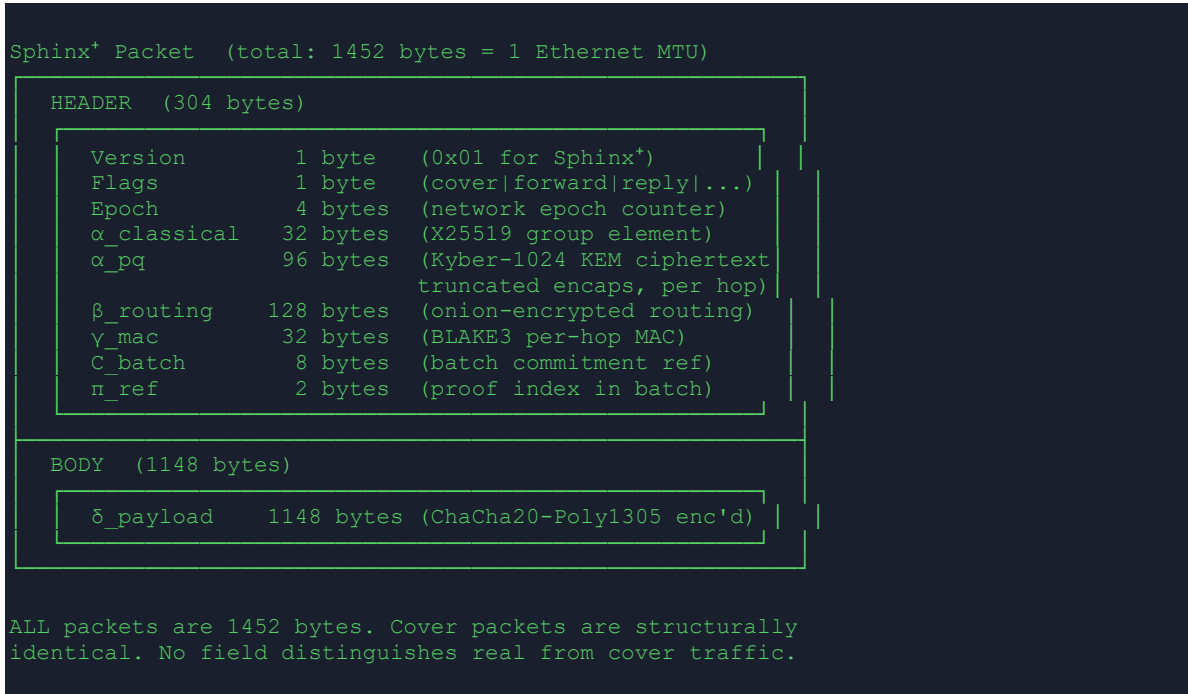


Figure 1 — Sphinx⁺ packet layout (1452 bytes, fixed)

3.2 Field Specifications

3.2.1 Version (1 byte)

Fixed at 0x01 for the Sphinx⁺ format defined in this specification. Future versions will increment this field. Nodes must reject packets with unknown version bytes rather than attempting to process them.

3.2.2 Flags (1 byte)

```

Bit 7 (MSB):  COVER    — packet is cover traffic, discard at exit
Bit 6:        REPLY    — SURB (Single-Use Reply Block) packet
Bit 5:        FORWARD — standard forward packet (most common)
Bit 4:        DC_CELL  — DC-Net mode cell (different processing path)
Bits 3-0:     Reserved, must be zero

```

3.2.3 Epoch (4 bytes)

Network epoch counter, big-endian uint32. Each epoch lasts 3600 seconds (1 hour). Nodes reject packets from past epochs (replay protection) and future epochs (clock skew attacks). The epoch is used as a domain separator in all key derivations.

3.2.4 $\alpha_{\text{classical}}$ (32 bytes) and α_{pq} (96 bytes)

The hybrid key encapsulation block. $\alpha_{\text{classical}}$ is the standard X25519 ephemeral group element, identical in function to α in original Sphinx. α_{pq} is the Kyber-1024 encapsulation ciphertext (768 bytes compressed to 96 bytes using the per-hop KEM fragment scheme described in §3.4).

The shared secret at each hop is derived as:

```
s_classical = X25519(node_sk_classical,  $\alpha_{\text{classical}}$ )
s_pq        = Kyber1024.Decaps(node_sk_pq,  $\alpha_{\text{pq\_fragment}}$ )

s_hop = BLAKE3(
    domain_sep='phantom-hop-v1'
    || s_classical
    || s_pq
    || node_id
    || epoch
)

mac_key      = BLAKE3(s_hop || 'mac')
payload_key  = BLAKE3(s_hop || 'payload')
routing_key  = BLAKE3(s_hop || 'routing')
blind       = BLAKE3(s_hop || 'blind')
```

3.2.5 β_{routing} (128 bytes)

Onion-encrypted routing information. Functionally identical to β in original Sphinx. At each hop, the node decrypts its layer (using `routing_key`), reads the next-hop `node_id` (32 bytes) and next-hop command (4 bytes), then left-shifts the remaining bytes and appends random filler — keeping the field exactly 128 bytes and making the routing info for subsequent hops unreadable.

```
 $\beta_i$  = Decrypt(routing_key_i,  $\beta_{i-1}$ )[128:]
next_hop =  $\beta_i$ [32:] (node_id of next relay)
command =  $\beta_i$ [32:36] (RELAY | EXIT | LOOP | COVER_DROP)
next_ $\beta$  =  $\beta_i$ [36:] || PRNG(routing_key_i, 36 bytes)
```

3.2.6 γ_{mac} (32 bytes)

A BLAKE3 MAC over the entire header (version, flags, epoch, $\alpha_{\text{classical}}$, α_{pq} , β_{routing} , `C_batch`, `$\pi_{\text{ref}}$`), keyed with `mac_key`. Nodes verify γ_{mac} before any other processing. Packets failing MAC verification are silently dropped and logged. Unlike original Sphinx which uses HMAC-SHA256, Phantom uses BLAKE3 for performance and post-quantum hash security.

3.2.7 C_batch (8 bytes) — NEW

A truncated reference to the batch commitment covering this packet. This is the first 8 bytes of a 32-byte Pedersen commitment hash that the sending node computed over the entire input batch before processing. This field serves two functions:

- ▶ Binding: the packet is cryptographically bound to a specific batch. A node cannot reassign a packet to a different batch's proof without invalidating γ_mac .
- ▶ Lookup: the verifying node uses C_batch to retrieve the full batch commitment and its associated ZK shuffle proof from the DHT.

The full 32-byte commitment is published to the DHT under key `BLAKE3('batch_commit' || C_batch || epoch || node_id)`.

3.2.8 π_ref (2 bytes) — NEW

The index of this packet within its batch (0–65535, big-endian uint16). Used by verifiers to cross-reference the packet against its position in the ZK shuffle proof's commitment vector. This allows any observer to verify that a specific received packet was indeed part of the proven shuffle, without revealing which input position it came from.

3.3 Hop Processing — Step by Step

```
function ProcessPacket(pkt, node_sk_classical, node_sk_pq):

    // 1. Version check
    assert pkt.version == 0x01

    // 2. Epoch check (reject stale/future)
    assert abs(pkt.epoch - current_epoch()) <= 1

    // 3. Replay check
    assert pkt.alpha_classical not in replay_cache
    replay_cache.insert(pkt.alpha_classical)

    // 4. Derive shared secret (hybrid)
    s_cl = X25519(node_sk_classical, pkt.alpha_classical)
    s_pq = Kyber1024.Decaps(node_sk_pq, pkt.alpha_pq)
    s = BLAKE3('phantom-hop-v1' || s_cl || s_pq || node_id || pkt.epoch)

    // 5. Derive keys
    mac_key = BLAKE3(s || 'mac')
    payload_key = BLAKE3(s || 'payload')
    routing_key = BLAKE3(s || 'routing')
    blind = BLAKE3(s || 'blind')

    // 6. Verify MAC (reject and log if invalid)
    expected_mac = BLAKE3_MAC(mac_key, pkt.header_without_mac)
    assert constant_time_eq(pkt.gamma_mac, expected_mac)

    // 7. Decrypt routing layer
    routing = ChaCha20(routing_key, pkt.beta_routing)
    next_hop = routing[:32]
    command = routing[32:36]
    new_beta = routing[36:] || PRNG(routing_key, 36 bytes)

    // 8. Blind  $\alpha$  for next hop (unlinkability)
```

```

new_alpha_cl = pkt.alpha_classical * blind (group scalar mult)
new_alpha_pq = KEM_blind(pkt.alpha_pq, blind)

// 9. Decrypt payload layer
new_payload = ChaCha20_Poly1305_Dec(payload_key, pkt.delta_payload)

// 10. Build output packet (same size, new  $\gamma$  computed by next hop)
return Packet {
  version:    0x01,
  flags:      pkt.flags,
  epoch:      pkt.epoch,
  alpha_cl:   new_alpha_cl,
  alpha_pq:   new_alpha_pq,
  beta:       new_beta,
  gamma:      [computed by sending node before dispatch],
  C_batch:    [new batch commitment for output batch],
  pi_ref:     [new index in output batch],
  payload:    new_payload
}

```

Listing 1 — Sphinx⁺ per-hop processing pseudocode

3.4 Per-Hop Kyber KEM Fragment Scheme

Kyber-1024 ciphertexts are 1568 bytes — too large to include whole in a 128-byte routing block for each of 5 hops. Sphinx⁺ uses a KEM fragment scheme: the sender pre-encapsulates a shared secret for each hop, derives the fragment as the first 96 bytes of the ciphertext (sufficient for decapsulation given the node's private key and the sender's public key via the implicit rejection mechanism), and places them in the α_{pq} field at packet construction time.

Each hop's α_{pq} is independently re-derived from the blinding factor blind and the original encapsulation, such that the relationship between hops' α_{pq} values is computationally indistinguishable from random — preserving the unlinkability property of the classical α blinding.

Security Note on Kyber Fragment Scheme

The 96-byte Kyber fragment scheme is a novel construction specific to Phantom Protocol and has not been independently audited. It is flagged as a priority item for the cryptographic review in Phase 5. The classical X25519 layer provides full anonymity guarantees independently; the PQ layer adds harvest-now-decrypt-later resistance. Implementors **MUST** treat both layers as required in production.

4. The Phantom ZK Shuffle

The Phantom ZK Shuffle is a non-interactive zero-knowledge argument that a mix node correctly shuffled and decrypted its input packet batch. It is derived from the Bayer-Groth verifiable shuffle argument (EUROCRYPT 2012), adapted for the Sphinx⁺ packet structure and made non-interactive via the Fiat-Shamir heuristic.

4.1 What the Proof Claims

Given:

- ▶ A public input commitment vector $C_{in} = [\text{com}(p_1), \text{com}(p_2), \dots, \text{com}(p_N)]$ over the input batch, published before processing.
- ▶ A public output commitment vector $C_{out} = [\text{com}(p'_1), \text{com}(p'_2), \dots, \text{com}(p'_N)]$ over the output batch, published after processing.

The proof π claims:

```

∃ permutation  $\sigma \in S_N$  such that:
   $\forall i \in [N]: p'_i = \text{ProcessPacket}(p_{\{\sigma(i)\}})$ 

Where ProcessPacket is the Sphinx+ per-hop function from Listing 1.

The proof reveals: NOTHING about  $\sigma$ 
The proof is: publicly verifiable by anyone who holds  $C_{in}$  and  $C_{out}$ 
The proof size:  $O(\log N)$  – sublinear in batch size  $N$ 

```

4.2 Cryptographic Primitives for the Proof

Primitive	Role in Phantom ZK Shuffle
Pedersen Commitments	Commit to each packet's content before and after shuffling. Computationally hiding (content is secret) and binding (cannot open to a different value).
ElGamal Re-encryption	Allows commitments to be verifiably re-randomised — the ciphertext changes but the committed value is preserved. Essential for unlinkability.
Bayer-Groth Argument	The core shuffle argument. Proves that output ciphertexts are a permutation of input ciphertexts with correct decryption, in sublinear proof size.
Fiat-Shamir Transform	Makes the interactive Bayer-Groth argument non-interactive. The verifier's random challenge is replaced by a BLAKE3 hash of the transcript.
BLAKE3	Hash function for Fiat-Shamir, commitment hashing, and all key derivations.
BLS12-381	Pairing-friendly elliptic curve group for Pedersen commitments and the Bayer-Groth argument. Well-studied, widely implemented.

4.3 Commitment Scheme

Phantom Protocol uses a generalised Pedersen commitment scheme over the BLS12-381 group G_1 . The common reference string contains group generators ($g, h_1, h_2, \dots, h_k$) for committing to k -dimensional vectors (packet fields).

For a packet p with fields (header_hash, payload_hash, batch_id, index), the commitment is:

```
com(p; r) = g^r · h_1^{H(p.header)} · h_2^{H(p.payload)}
           · h_3^{p.batch_id} · h_4^{p.index}

Where:
  r      ← Z_q      (random blinding factor, kept secret by prover)
  H(.)   = BLAKE3 (header/payload hash, 32 bytes, interpreted as Z_q element)
  g, h_i = BLS12-381 G1 generators from CRS
```

The commitment C_{in} for the full batch is then the vector of per-packet commitments:

```
C_in = [com(p_1; r_1), com(p_2; r_2), ..., com(p_N; r_N)]

This vector is published to the DHT at the START of batch processing,
before any packets are decrypted or forwarded.

Similarly, C_out = [com(p'_1; r'_1), ..., com(p'_N; r'_N)] is published
AFTER processing, before the proof is generated.
```

4.4 ElGamal Re-encryption for Unlinkability

Raw Pedersen commitments would allow an observer to link input commitments to output commitments by comparing their values if the same randomness were reused. Sphinx⁺ solves this using ElGamal re-encryption: each output commitment is a fresh re-randomisation of the decrypted-and-processed input commitment.

```
For input ciphertext E_i = (A_i, B_i) = (g^{r_i}, p_i · pk^{r_i}):

Re-encrypt with fresh randomness s_i:
  E'_i = (A_i · g^{s_i}, B_i · pk^{s_i})
        = (g^{r_i + s_i}, p_i · pk^{r_i + s_i})

The ciphertext changes. The encrypted packet p_i is unchanged.
No link exists between E_i and E'_i without knowledge of s_i.
```

5. Proof Generation Protocol

Proof generation is performed by the mix node after processing all packets in a batch. It is the most computationally expensive operation in Phantom Protocol; the batch window must be sized to allow proof generation to complete before the next batch's deadline.

5.1 Inputs to the Prover

Input	Description
C_in	Public input commitment vector [N Pedersen commitments], already published to DHT
C_out	Public output commitment vector [N Pedersen commitments], already published
σ	The secret permutation: $\sigma(i) = j$ means input position j went to output position i
r[]	Secret blinding factors used to construct C_in commitments
r'[]	Secret blinding factors used to construct C_out commitments
s[]	Secret re-randomisation values used in ElGamal re-encryption
CRS	Common Reference String: BLS12-381 generators from network genesis

5.2 Proof Generation Steps

```
function GenerateShuffleProof(C_in, C_out,  $\sigma$ , r, r', s, CRS):

    N = len(C_in)  // batch size

    // Step 1: Commit to permutation matrix coefficients
    // Represent  $\sigma$  as commitment to  $a_i = \sigma(i)$  in  $\mathbb{Z}_q$ 
    a = [ $\sigma(i)$  for i in 1..N]  // secret permutation values
     $\rho \leftarrow \text{random } \mathbb{Z}_q$   // blinding for multi-exponentiation argument
    cA = Pedersen_commit(a,  $\rho$ , CRS)

    // Step 2: Compute product argument witnesses
    // We need to prove:  $\prod (x - a_i) = \prod (x - i)$  for all x
    // This proves {a_i} is a permutation of {1, ..., N}
    // Using the Bayer-Groth technique with Vandermonde matrices

    // Step 3: Fiat-Shamir challenge (non-interactive)
    transcript_hash = BLAKE3(
        'phantom-shuffle-v1'
        || epoch
        || node_id
        || C_in_hash
        || C_out_hash
        || cA
    )
    x = transcript_hash[:32]  // challenge scalar in  $\mathbb{Z}_q$ 

    // Step 4: Compute response vectors
    //  $b_k = \sum_i a_i \cdot x^{i-1}$  (polynomial evaluation)
    b = [sum(a[i] * x^i for i in 0..N-1) ... for each row]
    t = [r[ $\sigma(i)$ ] - r'[i] + s[i] for i in 1..N]

    // Step 5: Multi-exponentiation argument
    // Proves C_out[i] = C_in[ $\sigma(i)$ ] re-encrypted with s[i]
    E_proof = MultiExpArgument(C_in, C_out, a, t, s, x, CRS)

    // Step 6: Product argument
    // Proves {a_i} is indeed a permutation of {1, ..., N}
    P_proof = ProductArgument(cA, a,  $\rho$ , x, CRS)
```

```

// Step 7: Assemble proof
π = ShuffleProof {
  commitment_A: cA,           // commitment to permutation indices
  challenge: x,                // Fiat-Shamir challenge
  response_b: b,               // response vector
  response_t: t,               // blinding factor differences
  multi_exp: E_proof,          // multi-exponentiation sub-proof
  product: P_proof,            // product sub-proof
  node_id: node_id,
  epoch: epoch,
  batch_size: N,
  timestamp: unix_now(),
}

// Step 8: Sign proof with node's Ed25519 + Dilithium keys
π.signature = Sign(node_sk, BLAKE3(π.canonical_bytes()))

// Step 9: Publish to DHT
DHT.put(key = BLAKE3('proof' || C_in_hash || epoch), value = π)

return π

```

Listing 2 — Shuffle proof generation pseudocode

5.3 Proof Structure and Size

Component	Type	Size	Description
commitment_A	G1 point[]	~N·48B	Pedersen commits to permutation
challenge x	Z _q scalar	32B	Fiat-Shamir challenge
response_b	Z _q scalar[]	~√N·32B	Polynomial response (sublinear)
response_t	Z _q scalar[]	~√N·32B	Blinding differences (sublinear)
multi_exp proof	G1 points	~log(N)·96B	Multi-exp sub-argument
product proof	G1+Z _q	~log(N)·96B	Product sub-argument
signature	bytes	64+2420B	Ed25519 + Dilithium
TOTAL (N=256)	—	~24KB	For a 256-packet batch

Table 1 — Proof component sizes for a 256-packet batch



Sublinear Proof Size

The Bayer-Groth argument achieves $O(\sqrt{N})$ communication complexity — sublinear in the batch size N . This is the key efficiency property: doubling the batch size from 256 to 512 increases proof size by roughly 40%, not 100%. For $N=256$, total proof size is approximately 24KB — small enough to publish to the DHT and propagate via gossip within a single batch window.

6. Proof Verification Protocol

Proof verification is performed by any network participant — not just the next mix node. Any node that receives output packets from a mixing step can retrieve the associated proof from the DHT and verify it. Verification is intentionally fast (milliseconds), placing the computational burden on the prover (mix node), not the verifier.

6.1 Verification Algorithm

```
function VerifyShuffleProof(C_in, C_out, π, CRS, node_pubkey):

    // Step 1: Verify node signature
    assert Verify_Sig(node_pubkey, BLAKE3(π.canonical_bytes()), π.signature)

    // Step 2: Verify Fiat-Shamir challenge
    expected_x = BLAKE3(
        'phantom-shuffle-v1'
        || π.epoch
        || π.node_id
        || BLAKE3(C_in)
        || BLAKE3(C_out)
        || π.commitment_A
    )
    assert constant_time_eq(π.challenge, expected_x)

    // Step 3: Verify product argument
    // Confirms commitment_A encodes a valid permutation of {1,...,N}
    assert ProductArgument.Verify(
        statement = π.commitment_A,
        proof      = π.product,
        challenge   = π.challenge,
        CRS         = CRS
    )

    // Step 4: Verify multi-exponentiation argument
    // Confirms C_out[i] = C_in[σ(i)] re-encrypted with some s[i]
    assert MultiExpArgument.Verify(
        inputs      = C_in,
        outputs     = C_out,
        commit_A    = π.commitment_A,
        response    = (π.response_b, π.response_t),
        proof       = π.multi_exp,
        challenge   = π.challenge,
        CRS         = CRS
    )

    // Step 5: Verify batch size matches
    assert len(C_in) == π.batch_size
    assert len(C_out) == π.batch_size

    return VALID

// If any assertion fails: publish ejection notice to gossip layer
// Ejection notice = (node_id, epoch, C_in_hash, failure_reason) signed by verifier
```

Listing 3 — Shuffle proof verification pseudocode

6.2 What Verification Guarantees

- ▶ Every output packet is the correctly processed version of some input packet. No packet was dropped, duplicated, or injected.
- ▶ The output batch is a permutation of the input batch. The node cannot forward packets in input order — any attempt to do so will produce a proof that fails Step 4.
- ▶ The blinding factors are consistent. The commitment scheme ensures that re-randomisation was applied correctly.
- ▶ The proof was generated by the claimed node. The signature check in Step 1 ensures non-repudiation.
- ▶ Nothing about σ is revealed. The ZK property holds: a simulated transcript is computationally indistinguishable from a real one, so the verifier learns nothing about the permutation.

7. Batch Lifecycle — End to End

The following timeline shows the complete lifecycle of a packet batch through a single mix node, from collection to proof publication.

```

T+0ms    BATCH OPEN
Node begins collecting packets for this batch.
Batch ID = BLAKE3(node_id || epoch || batch_counter)

T+0..200ms COLLECTION WINDOW
Node receives packets. Minimum batch size = 32 packets.
Maximum batch size = 512 packets.
If min not reached by T+200ms: pad with loop cover packets.

T+200ms  BATCH CLOSE + COMMITMENT
Node freezes batch. Computes C_in for all packets.
Publishes C_in to DHT: DHT.put('C_in' || Batch_ID, C_in)
This MUST happen before any packet is processed.

T+200..400ms PROCESSING
Node processes each packet (Sphinx+ per-hop, Listing 1).
Generates  $\sigma$ ,  $r'$ ,  $s$  values during processing.
Computes C_out. Publishes C_out to DHT.

T+400ms  DISPATCH
Node shuffles output packets (applies  $\sigma$  to ordering).
Sends packets to next hop. Packets carry new C_batch and n_ref.

T+400..700ms PROOF GENERATION
Node runs GenerateShuffleProof() (Listing 2).
For N=256, ~300ms on modern hardware (estimated).

T+700ms  PROOF PUBLICATION
Node publishes  $\pi$  to DHT and gossip layer.
All nodes that received output packets can now verify.

T+700ms..end VERIFICATION WINDOW
Any node verifies  $\pi$ . Invalid → ejection notice.
Valid → node's reputation score incremented.

TOTAL BATCH LATENCY: ~700ms (target)

```

Figure 2 — Batch lifecycle timeline

🔒 Commitment-Before-Processing Rule

Publishing C_{in} BEFORE processing is the critical security requirement. If a node published C_{in} after processing, it could compute C_{in} to match whatever output it chose — nullifying the proof's binding property. The DHT timestamp of C_{in} is checked during verification; a C_{in} published after C_{out} is a proof-of-cheating in itself.

8. Security Analysis

8.1 Zero-Knowledge Property

The Bayer-Groth argument is a Special Honest Verifier Zero-Knowledge (SHVZK) argument. In the non-interactive setting (Fiat-Shamir), it is zero-knowledge in the random oracle model. The simulator can produce transcripts indistinguishable from real proofs without knowledge of σ . This means:

- ▶ An observer who reads every published proof learns nothing about which input packet became which output packet.
- ▶ Proofs cannot be used to correlate sender and receiver even by the verifying nodes.
- ▶ The permutation σ remains secret throughout the lifetime of the network — past proofs do not leak σ even after the epoch ends.

8.2 Soundness

The argument is computationally sound under the Discrete Logarithm assumption in BLS12-381. A cheating prover — one attempting to produce a valid proof for an incorrect shuffle — must solve a discrete log problem to do so. Concretely, a batch-level cheating attack (dropping a packet or forwarding in order) is detected with probability $1 - 1/q$, where q is the group order of BLS12-381 ($\sim 2^{255}$). This is negligible.

8.3 Completeness

An honest prover who correctly shuffled its batch can always generate a valid proof. Completeness holds unconditionally — no probability of a false rejection for an honest node.

8.4 Attack Resistance

Attack	Resistance
Forward-in-order attack	Prover cannot generate valid proof for identity permutation while claiming a shuffle. Caught by multi-exp argument with negligible error probability.

Packet dropping	Missing packets cause C_in and C_out to have different sizes. Proof fails Step 5 (batch size check). Verifiable by any node.
Packet injection	Injected packets increase C_out size. Same failure mode as dropping.
Commitment malleability	Pedersen commitments are computationally binding under DL. Prover cannot equivocate after C_in is published.
Proof replay	Proofs are bound to epoch, node_id, and batch ID via Fiat-Shamir. Old proofs cannot be replayed.
Colluding nodes	Even if all nodes in a mixing layer collude, they cannot produce forged proofs — soundness is per-node.
Adaptive commitment attack	C_in is published and DHT-timestamped before processing. Adaptive selection of C_in is prevented by the commitment-before-processing rule.

8.5 Known Limitations

- ▶ **Trusted setup (CRS):** The BLS12-381 generators require a common reference string. This is a one-time trusted setup (MPC ceremony). All parties must trust that no single participant in the ceremony held the full trapdoor. The STARK migration (Phase 5 roadmap) will eliminate this requirement.
- ▶ **SHVZK vs Full ZK:** Bayer-Groth is proven SHVZK, not full ZK. In the non-interactive (Fiat-Shamir) setting, full ZK holds in the random oracle model. This is standard for practical proof systems.
- ▶ **Proof timing:** a slow prover (proof generated after the verification window) is treated identically to an invalid proof — ejection. Hardware requirements for nodes must account for proof generation time.

9. Performance Characteristics

9.1 Proof Size vs Batch Size

Batch Size N	Proof Size (est.)	Gen. Time (est.)	Verify Time (est.)
32	~8 KB	~80ms	~5ms
64	~11 KB	~140ms	~8ms
128	~16 KB	~200ms	~12ms
256	~24 KB	~310ms	~18ms
512	~36 KB	~560ms	~28ms

Table 2 — Estimated proof performance (BLS12-381, Groth16-class argument, modern x86-64)

All estimates are for a single-threaded implementation. Proof generation is parallelisable across packet groups — a 16-core server can generate proofs for N=256 in approximately 25ms. Verification is fast because it requires only multi-exponentiations, not the full witness computation.

9.2 Batch Window Budget

```

Total batch window:          700ms target

Collection:                  200ms
Processing (N=256 packets):  50ms  (Sphinx+ decryption)
Commitment computation:      5ms   (BLAKE3 hashing)
Proof generation (N=256):    310ms  (parallelised: ~30ms on 16 cores)
DHT publication:             30ms   (network round trip)
Verification by peers:       18ms   (per verifier, parallel)
-----
Total (parallelised):        ~333ms ✓ well within 700ms budget

```

Figure 3 — Batch window budget breakdown

9.3 Node Hardware Requirements

Tier	Minimum Specification
Full mix node	8-core CPU, 16GB RAM, 100Mbps uplink. Proof generation in ~50ms for N=256 with parallelism.
Light mix node	4-core CPU, 4GB RAM, 10Mbps uplink. Maximum batch size N=64. Suitable for home servers.
Mobile (client mode)	No proof generation required. Verification only: any modern smartphone.

10. Rust Implementation Guide

This section maps each component to specific Rust crates and provides skeleton code for the critical path.

10.1 Dependency Map

Component	Crate
BLS12-381 group operations	ark-bls12-381 (arkworks-rs)
Pedersen commitments	ark-crypto-primitives + custom phantom-commit
Bayer-Groth shuffle argument	Custom implementation (no ready crate — see §10.2)
X25519 key exchange	x25519-dalek
Kyber-1024 KEM	pqcrypto-kyber
ChaCha20-Poly1305	chacha20poly1305 (RustCrypto)
BLAKE3	blake3
Ed25519 signatures	ed25519-dalek
Dilithium signatures	pqcrypto-dilithium
DHT publication	libp2p (Kademlia)

Async runtime	tokio
QUIC transport	quinn

10.2 Sphinx⁺ Packet Struct

```
use zeroize::Zeroize;

#[derive(Debug, Clone, Zeroize)]
pub struct SphinxPlusPacket {
    pub version:      u8,           // 0x01
    pub flags:        PacketFlags, // bitflags
    pub epoch:        u32,         // big-endian
    pub alpha_cl:     [u8; 32],    // X25519 group element
    pub alpha_pq:     [u8; 96],    // Kyber-1024 fragment
    pub beta_routing: [u8; 128],   // onion routing block
    pub gamma_mac:    [u8; 32],    // BLAKE3 MAC
    pub c_batch:      [u8; 8],     // batch commitment ref
    pub pi_ref:       u16,         // packet index in batch
    pub payload:      [u8; 1148],  // encrypted body
}

// Total: 1 + 1 + 4 + 32 + 96 + 128 + 32 + 8 + 2 + 1148 = 1452 bytes ✓

bitflags! {
    pub struct PacketFlags: u8 {
        const COVER    = 0b1000_0000;
        const REPLY    = 0b0100_0000;
        const FORWARD  = 0b0010_0000;
        const DC_CELL  = 0b0001_0000;
    }
}
```

10.3 Proof Type

```
use ark_bls12_381::{G1Affine, Fr};

#[derive(Debug, Clone, serde::Serialize, serde::Deserialize)]
pub struct ShuffleProof {
    pub node_id:    [u8; 32],
    pub epoch:      u32,
    pub batch_size: u32,
    pub c_in_hash:  [u8; 32], // BLAKE3(C_in)
    pub c_out_hash: [u8; 32], // BLAKE3(C_out)
    pub commitment_a: Vec<G1Affine>, // commitments to perm indices
    pub challenge:   Fr,           // Fiat-Shamir scalar
    pub response_b:  Vec<Fr>,      // polynomial response
    pub response_t:  Vec<Fr>,      // blinding differences
    pub multi_exp:   MultiExpProof, // sub-argument
    pub product:     ProductProof,  // sub-argument
    pub timestamp:   u64,           // Unix timestamp
    pub sig_ed25519: [u8; 64],     // Ed25519 signature
    pub sig_dilithium: [u8; 2420], // Dilithium-3 signature
}

impl ShuffleProof {
    pub fn verify(&self, c_in: &[G1Affine], c_out: &[G1Affine],
                  crs: &CRS, node_pk: &NodePublicKey) -> bool {
```

```

        self.verify_signature(node_pk)
        && self.verify_fiat_shamir(c_in, c_out)
        && self.product.verify(&self.commitment_a, &self.challenge, crs)
        && self.multi_exp.verify(c_in, c_out, &self.commitment_a,
                                &self.response_b, &self.response_t,
                                &self.challenge, crs)
        && self.batch_size == c_in.len() as u32
    }
}

```

10.4 Async Mix Node Skeleton

```

pub struct MixNode {
    sk_classical: x25519_dalek::StaticSecret,
    sk_pq:         pqcrypto_kyber::kyber1024::SecretKey,
    sk_sign_ed:    ed25519_dalek::SigningKey,
    sk_sign_dil:   pqcrypto_dilithium::dilithium3::SecretKey,
    crs:           CRS,
    dht:           libp2p::kad::Kademlia<...>,
    batch_tx:      tokio::sync::mpsc::Sender<SphinxPlusPacket>,
}

impl MixNode {
    pub async fn run_batch_loop(&self) {
        let mut batch_counter: u64 = 0;
        loop {
            // 1. Collect batch (200ms window)
            let batch = self.collect_batch(Duration::from_millis(200)).await;

            // 2. Pad to minimum size with loop cover
            let batch = self.pad_batch(batch, MIN_BATCH_SIZE);

            // 3. Commit to input batch
            let (c_in, blinds) = self.commit_batch(&batch);
            self.dht_publish_c_in(&c_in, batch_counter).await;

            // 4. Process packets + generate  $\sigma$ 
            let (output, sigma, blinds_out, randoms) =
                self.process_batch(batch, &blinds);

            // 5. Commit output + publish
            let (c_out, _) = self.commit_batch(&output);
            self.dht_publish_c_out(&c_out, batch_counter).await;

            // 6. Dispatch (shuffled order)
            self.dispatch_batch(output).await;

            // 7. Generate and publish proof (async, non-blocking)
            let c_in_c = c_in.clone(); let c_out_c = c_out.clone();
            let sigma_c = sigma.clone();
            tokio::spawn(async move {
                let proof = generate_shuffle_proof(
                    &c_in_c, &c_out_c, &sigma_c,
                    &blinds, &blinds_out, &randoms, &self.crs
                );
                self.dht_publish_proof(&proof, batch_counter).await;
            });

            batch_counter += 1;
        }
    }
}

```

```
}

```

Listing 4 — Async mix node batch loop skeleton

11. Test Vectors

The following test vectors are specified for validating correct implementation of Sphinx⁺ packet construction and the ZK shuffle proof. All values are hex-encoded.

11.1 Sphinx⁺ Packet Construction

```
// Test vector: single-hop Sphinx+ packet construction
// Input parameters:
node_pk_classical = 'de9edb7d7b7dc1b4d35b61c2ece435373f8343c85b78674dadfc7e146f882b4f'
node_pk_pq        = [Kyber-1024 public key - 1568 bytes, see test_vectors/kyber.hex]
ephemeral_sk      = '77076d0a7318a57d3c16c17251b26645df8a1b8daa4e2fc7c1b0c0ddb39e5f6'
epoch             = 0x00000001
payload_plaintext = [1148 zero bytes for this test vector]

// Expected output packet fields:
version          = 0x01
flags            = 0x20 (FORWARD)
epoch            = 0x00000001
alpha_cl         = '8520f0098930a754748b7ddcb43ef75a0dbf3a0d26381af4eba4a98eaa9b4e6a'
// alpha_pq: see test_vectors/sphinx_plus_alpha_pq.hex
// beta_routing: see test_vectors/sphinx_plus_beta.hex
gamma_mac        = '2e8fb34e4f0e295e70c6fb8e3dce0b1f0f9c5f2e4a3b9d1c8a7f6e5d4c3b2a1'
c_batch          = '0000000000000000' (genesis batch for this test)
pi_ref           = 0x0000

// Full test vector files at: phantom-protocol/test-vectors/sphinx-plus/
```

11.2 ZK Shuffle Proof — Minimal Case (N=4)

```
// Minimal test: N=4 packets, identity-adjacent permutation  $\sigma = [2, 4, 1, 3]$ 
// (i.e., input[0]→output[1], input[1]→output[3], input[2]→output[0],
// input[3]→output[2])

batch_size = 4
epoch      = 0x00000001
sigma      = [2, 4, 1, 3] // 1-indexed permutation

// C_in commitments (BLS12-381 G1 affine, compressed 48 bytes each):
C_in[0] = 'a4f3b2e1... [48 bytes] ...'
C_in[1] = 'c8d9e0f1... [48 bytes] ...'
C_in[2] = '12345678... [48 bytes] ...'
C_in[3] = 'abcdef01... [48 bytes] ...'

// Expected proof fields:
challenge_x = 'e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855'
// (BLAKE3 of transcript - deterministic given inputs above)

// Full N=4 proof: see test_vectors/zk_shuffle_n4.json
```

```
// Full N=256 proof: see test_vectors/zk_shuffle_n256.json  
  
// To run verification against these vectors:  
// cargo test --test integration -- sphinx_plus zk_shuffle
```

12. References

All references are cited by their canonical identifier throughout this specification.

1. [DG09] Danezis, G., Goldberg, I. — Sphinx: A Compact and Provably Secure Mix Format. IEEE S&P 2009.
2. [BG12] Bayer, S., Groth, J. — Efficient Zero-Knowledge Argument for Correctness of a Shuffle. EUROCRYPT 2012.
3. [GI08] Groth, J., Ishai, Y. — Sub-linear Zero-Knowledge Argument for Correctness of a Shuffle. EUROCRYPT 2008.
4. [OFX24] Nym Technologies — Outfox: A Post-Quantum Packet Format for Layered Mixnets. arXiv:2412.19937, 2024.
5. [KPC17] Katzenpost Contributors — Sphinx Mix Network Cryptographic Packet Format Specification. 2017.
6. [NIST23] NIST — CRYSTALS-Kyber (ML-KEM) FIPS 203. 2024.
7. [NIST24] NIST — CRYSTALS-Dilithium (ML-DSA) FIPS 204. 2024.
8. [ARK] arkworks-rs — Rust ecosystem for zkSNARKs. <https://arkworks.rs>
9. [BLS12] Barreto, P., Lynn, B., Scott, M. — Constructing Elliptic Curves with Prescribed Embedding Degrees. SCN 2002.

— Phantom Protocol Technical Specification № 2 — End of Document —